Phase 2: Server Additions

Goal: Add Lightning payout, timelock escape path, and verify audit integrity — on top of the upgraded existing module. **Prerequisite:** Phase 1 complete (existing module upgraded to v0.11, all tests pass) **Estimated effort:** 1 week

---

Context for Claude

The existing module (from Phase 1) already implements: - `process_output`: locks ecash, stores contract in DB - `process_input` (ClamingWithoutDispute): seller claims with secret code - `process_input` (Disputing): buyer or seller initiates dispute - `process_input` (ArbiterDecision): single arbiter decides - `process_input` (ClaimingAfterDispute): winner claims ecash

What the existing module does NOT have and we must add: 1. **Lightning payout** — ecash goes directly to the winning party now; we need LN invoice submission and payout 2. **Timelock escape path** — if service disappears, users must be able to recover funds 3. **Audit** was commented out — re-enabled in Phase 1, but verify it's correct here

The Nostr oracle replacement (Phase 3) is NOT part of this phase. The single arbiter stays for now.

Critical invariant: Every ecash entering via `process_output` creates an audit liability. Every ecash leaving via `process_input` removes that liability. At all times: `sum(audit items) = 0`. Violating this will cause the federation's audit to fail.

What Already Exists vs What to Add

Component	Status after Phase 1	Phase 2 action
`process_output` (lock funds)	Working	Verify audit item created, no changes
`process_input` (secret code)	Working	No changes
`process_input` (dispute)	Working	No changes
`process_input` (arbiter decision)	Working (single arbiter)	Leave for Phase 3
`process_input` (claim after dispute)	Working	No changes
`process_input` (timeout)	Missing	Add here
LN payout on release	Missing	Add here
`audit()`	Re-enabled in Phase 1	Verify correctness

Tasks

Task 2.0 — Understand existing database schema

Before adding anything, read the existing `db.rs` to understand what's already stored:

```
cat /home/ralf/fedimint-escrow/fedimint-escrow-server/src/db.rs
```

Map out which `DbKeyPrefix` values are already used (to avoid collisions when adding new ones).

Task 2.1 — Add timeout fields to EscrowOutput

The existing `EscrowOutput` struct needs two new fields. In `fedimint-escrow-common/src/lib.rs`:

```rust
pub struct EscrowOutput {
    pub amount: Amount,
    pub buyer_pubkey: PublicKey,
    pub seller_pubkey: PublicKey,
    pub arbiter_pubkey: PublicKey,
    pub escrow_id: String,
    pub secret_code_hash: String,
    pub max_arbiter_fee: Amount,
    // New fields:
    pub timeout_block: u32,       // block height after which escape is available
    pub timeout_action: TimeoutAction,  // who gets funds on timeout
}

pub enum TimeoutAction {
    Release,   // funds go to seller on timeout (default: seller delivered, buyer unresponsive,
    Refund,    // funds go to buyer on timeout (default: buyer paid, seller unresponsive)
}
```

Note: `timeout_block` and `TimeoutAction` need `Encodable`, `Decodable`, `Serialize`, `Deserialize` derives.

Task 2.2 — Add PayoutInvoice to additional database keys

In `fedimint-escrow-server/src/db.rs`, add the new key type for LN payout invoices (check existing prefix values to avoid collision):

```rust
// New addition only - do not repeat existing keys:
pub struct PayoutInvoiceKey(pub EscrowId);
pub struct EscrowTimeoutKey(pub EscrowId);  // stores timeout_block for scanning
```

Task 2.3 — Add Timeout input variant

In `fedimint-escrow-common/src/lib.rs`, add a new variant to `EscrowInput`:

```rust
pub enum EscrowInput {
    ClamingWithoutDispute(EscrowInputClamingWithoutDispute),
    Disputing(EscrowInputDisputing),
    ClaimingAfterDispute(EscrowInputClaimingAfterDispute),
    ArbiterDecision(EscrowInputArbiterDecision),
    // New:
    TimeoutClaim(EscrowInputTimeoutClaim),
}

pub struct EscrowInputTimeoutClaim {
    pub amount: Amount,
    pub escrow_id: String,
    pub hashed_message: [u8; 32],
    pub signature: Signature,
}
```

Task 2.4 — Implement Timeout path in process_input

In `fedimint-escrow-server/src/lib.rs`, add the timeout arm:

```rust
EscrowInput::TimeoutClaim(input) => {
    let escrow = /* load from DB */;

    // 1. Verify current block height >= timeout_block
    let current_height = /* get from consensus */;
    if current_height < escrow.timeout_block as u64 {
        return Err(EscrowInputError::TimelockNotExpired);
    }

    // 2. Determine who can claim based on timeout_action
    let expected_pubkey = match escrow.timeout_action {
        TimeoutAction::Release => escrow.seller_pubkey,
        TimeoutAction::Refund  => escrow.buyer_pubkey,
    };

    // 3. Verify signature
    // ... same pattern as existing process_input arms ...

    // 4. Finalize (remove contract + audit item)
}
```

Task 2.5 — Add Lightning payout submission endpoint

The existing module pays ecash directly to the winner. We need to add an LN invoice submission step so the winner receives Lightning instead of raw ecash.

This is a two-part change:

Part A: Store the payout invoice

Add a new `PayoutInvoiceKey` to the DB. The winning party submits a BOLT11 invoice (via the service API, not via Fedimint directly). The service stores it and triggers payout after the Fedimint transaction confirms.

Part B: Trigger LN payout after finalization

In the service backend (`deals.py`), after the Fedimint escrow is finalized, call `lnd_client.pay_invoice(bolt11)`. The LN payout flow is already implemented in the existing service — this connects it to the Fedimint confirmation event.

Note: The LN payout logic lives in the service layer (Python), not in the Fedimint module (Rust). The Fedimint module only handles ecash. The service listens for escrow finalization events and triggers LN payout.

Task 2.6 — Verify audit correctness

The `audit()` function was uncommented in Phase 1. Verify it works correctly:

```
# After running tests, check audit output
cargo test audit -- --nocapture
```

The invariant: after creating N escrows and releasing M of them, the audit sum must equal `(N - M) * amount`.

Write a specific audit test:

```
#[tokio::test]
async fn test_audit_balance() {
    // Create 3 escrows of 10_000 sats each → audit reports -30_000 msat
    // Release 2 of them → audit reports -10_000 msat
    // Release last one → audit reports 0
}
```

Task 2.7 — Update unit tests for new fields

The existing tests create `EscrowOutput` structs directly. After adding `timeout_block` and `timeout_action`, update all test constructions to include these fields. Tests should cover:

- Timeout claim BEFORE timeout block (expect: error)
- Timeout claim AFTER timeout block, correct key (expect: success)
- Timeout claim AFTER timeout block, wrong key (expect: error)

Task 2.N — Implement process_output verification (reference only)

The existing `process_output` already stores contracts and validates amounts. This is shown here for reference — do not rewrite, only extend if needed:

```rust
// Already implemented in the existing module:
// 1. Validate: amount > 0
// 2. Validate: escrow_id does not already exist
// 3. Store contract in database
// 4. Return amounts so Fedimint can verify balance
```

The only addition needed is storing the `timeout_block` in a secondary index for efficient scanning.

```rust
async fn process_output(
    &self,
    dbtx: &mut DatabaseTransaction<'_>,
    output: &EscrowOutput,
    out_point: OutPoint,
) -> Result<TransactionItemAmounts, EscrowOutputError> {

    // 1. Validate: amount > 0
    if output.contract.amount == Amount::ZERO {
        return Err(EscrowOutputError::ZeroAmount);
    }

    // 2. Validate: escrow_id does not already exist
    let existing = dbtx.get_value(&EscrowContractKey(output.contract.escrow_id)).await;
    if existing.is_some() {
        return Err(EscrowOutputError::DuplicateEscrowId);
    }

    // 3. Validate: 3 oracle pubkeys are present and non-duplicate
    // (validation logic here)

    // 4. Store contract in database
    dbtx.insert_entry(
        &EscrowContractKey(output.contract.escrow_id),
        &output.contract,
    ).await;

    // 5. Create audit liability (negative = we owe this to users)
    dbtx.insert_entry(
        &EscrowAuditKey(output.contract.escrow_id),
        &output.contract.amount,
    ).await;

    // 6. Return amounts so Fedimint can verify balance
    Ok(TransactionItemAmounts {
        amount: output.contract.amount,
        fee: self.cfg.fee_per_escrow,  // configurable service fee
    })
}
```

Task 2.3 — Implement process_input (unlock funds)

Called when funds are released or refunded.

```rust
async fn process_input(
    &self,
    dbtx: &mut DatabaseTransaction<'_>,
    input: &EscrowInput,
    in_point: InPoint,
) -> Result<InputMeta, EscrowInputError> {

    match input {
        EscrowInput::Cooperative { escrow_id, buyer_sig, seller_sig, beneficiary } => {
            let contract = self.get_contract(dbtx, escrow_id).await?;

            // Verify both signatures over the escrow_id + beneficiary
            let msg = cooperative_release_message(escrow_id, beneficiary);
            verify_sig(&contract.buyer_key, buyer_sig, &msg)?;
            verify_sig(&contract.seller_key, seller_sig, &msg)?;

            // Remove contract and audit item
            self.finalize_escrow(dbtx, escrow_id, contract.amount).await?;

            Ok(InputMeta { amount: contract.amount, .. })
        },

        EscrowInput::Timeout { escrow_id, beneficiary_sig } => {
            let contract = self.get_contract(dbtx, escrow_id).await?;

            // Verify current block height >= timeout_block
            let current_height = self.consensus_block_count(dbtx).await;
            if current_height < contract.timeout_block as u64 {
                return Err(EscrowInputError::TimelockNotExpired {
                    current: current_height,
                    required: contract.timeout_block as u64,
                });
            }

            // Determine beneficiary key from timeout_action
            let beneficiary_key = match contract.timeout_action {
                TimeoutAction::Release => contract.seller_key,
                TimeoutAction::Refund  => contract.buyer_key,
            };

            // Verify beneficiary signature
            let msg = timeout_claim_message(escrow_id);
            verify_sig(&beneficiary_key, beneficiary_sig, &msg)?;

            self.finalize_escrow(dbtx, escrow_id, contract.amount).await?;
```

```
            Ok(InputMeta { amount: contract.amount, .. })
        },

        EscrowInput::OracleAttestation { .. } => {
            todo!("Phase 3 - Nostr oracle integration")
        },
    }
}
```

Task 2.4 — Implement finalize_escrow helper

```
async fn finalize_escrow(
    &self,
    dbtx: &mut DatabaseTransaction<'_>,
    escrow_id: &EscrowId,
    amount: Amount,
) -> Result<()> {
    // Remove the contract
    dbtx.remove_entry(&EscrowContractKey(*escrow_id)).await;

    // Remove the audit liability (funds leaving the module)
    dbtx.remove_entry(&EscrowAuditKey(*escrow_id)).await;

    Ok(())
}
```

Task 2.5 — Implement audit

```
async fn audit(
    &self,
    dbtx: &mut DatabaseTransaction<'_>,
) -> Result<Vec<AuditItem>, AuditError> {
    // For each locked escrow, report a liability
    let mut items = vec![];
    let mut stream = dbtx.find_by_prefix(&EscrowAuditKeyPrefix).await;

    while let Some((key, amount)) = stream.next().await {
        items.push(AuditItem {
            name: format!("Escrow {}", key.0),
            milli_sat: -(amount.msats as i64),  // negative = liability
            module_instance_id: self.module_instance_id,
        });
    }
```

```
    Ok(items)
}
```

Task 2.6 — Unit tests

In `fedimint-escrow-server/src/tests.rs`:

```rust
#[tokio::test]
async fn test_create_escrow() {
    // Create output → verify contract stored in DB
    // Verify audit item created
}

#[tokio::test]
async fn test_cooperative_release_to_seller() {
    // Create escrow, then submit cooperative input with seller as beneficiary
    // Verify contract removed from DB
    // Verify audit item removed
}

#[tokio::test]
async fn test_cooperative_refund_to_buyer() {
    // Same as above but buyer as beneficiary
}

#[tokio::test]
async fn test_timeout_not_yet_expired() {
    // Submit timeout input before timeout block
    // Verify error: TimelockNotExpired
}

#[tokio::test]
async fn test_timeout_expired_release() {
    // Advance block height past timeout
    // Submit timeout input with seller sig
    // Verify success
}

#[tokio::test]
async fn test_wrong_signature_rejected() {
    // Submit cooperative input with wrong buyer sig
    // Verify error
}

#[tokio::test]
async fn test_audit_balance() {
    // Create 3 escrows, release 2, verify audit sum is correct
```

```
}
```

Definition of Done

- ☐ `EscrowOutput` has `timeout_block` and `timeout_action` fields
- ☐ `EscrowInput::TimeoutClaim` variant added
- ☐ `process_input` validates timeout path (correct sig + block height check)
- ☐ `audit()` verified correct: reports liabilities for all locked escrows
- ☐ LN payout trigger documented (service layer, not Fedimint module)
- ☐ Timeout tests: before/after timeout block, correct/wrong key
- ☐ Audit balance test: create N, release M, verify sum
- ☐ All tests pass: `cargo test`
- ☐ `cargo build` succeeds

Key References

- Existing module: `/home/ralf/fedimint-escrow/fedimint-escrow-server/src/lib.rs`
- Existing common types: `/home/ralf/fedimint-escrow/fedimint-escrow-common/src/lib.rs`
- Mint module audit pattern: `/home/ralf/fedimint/modules/fedimint-mint-server/src/lib.rs`
- LN module contract pattern: `/home/ralf/fedimint/modules/fedimint-ln-server/src/lib.rs`

Next: See 04_PHASE_3_ORACLE.md